

```

# =====
# STEP 1: GENERATE DATASET
# Global Urban Air Quality Index Dataset (2015-2025)
# =====

import pandas as pd
import numpy as np
import os
from datetime import datetime, timedelta

# Create folders
os.makedirs('dataset', exist_ok=True)
os.makedirs('outputs/charts', exist_ok=True)
os.makedirs('outputs/results', exist_ok=True)

np.random.seed(42)

# Cities with realistic AQI profiles
cities_data = [
    ('Delhi', 'India', 190, 45),
    ('Beijing', 'China', 165, 40),
    ('Dhaka', 'Bangladesh', 175, 42),
    ('Lahore', 'Pakistan', 185, 43),
    ('Mumbai', 'India', 150, 35),
    ('Shanghai', 'China', 145, 38),
    ('Karachi', 'Pakistan', 155, 36),
    ('Cairo', 'Egypt', 160, 37),
    ('Jakarta', 'Indonesia', 140, 33),
    ('Tehran', 'Iran', 155, 35),
    ('Mexico City', 'Mexico', 130, 30),
    ('Istanbul', 'Turkey', 115, 25),
    ('Bangkok', 'Thailand', 125, 28),
    ('Seoul', 'South Korea', 95, 22),
    ('Lagos', 'Nigeria', 135, 31),
    ('Los Angeles', 'USA', 75, 18),
    ('New York', 'USA', 65, 15),
    ('London', 'UK', 60, 14),
    ('Paris', 'France', 58, 13),
    ('Tokyo', 'Japan', 52, 12),
    ('Sydney', 'Australia', 48, 11),
    ('Toronto', 'Canada', 55, 13),
    ('Berlin', 'Germany', 53, 12),
    ('Singapore', 'Singapore', 70, 16),
    ('Sao Paulo', 'Brazil', 85, 20),
    ('Buenos Aires', 'Argentina', 78, 18),
    ('Moscow', 'Russia', 90, 21),
    ('Dubai', 'UAE', 110, 24),
    ('Nairobi', 'Kenya', 95, 22),
    ('Cape Town', 'South Africa', 72, 17)
]

# Generate data
start_date = datetime(2015, 1, 1)
end_date = datetime(2025, 12, 31)
weeks = (end_date - start_date).days // 7

records = []
print("Generating dataset... please wait...")

for week in range(weeks):
    current_date = start_date + timedelta(weeks=week)
    month = current_date.month
    num_cities = np.random.randint(20, 30)

```

```

num_cities = np.random.randint(20, 50)
selected = np.random.choice(len(cities_data), num_cities, replace=False)

for idx in selected:
    city, country, base_aqi, std_aqi = cities_data[idx]

    # Seasonal effect
    if month in [11, 12, 1, 2]:
        seasonal = 1.35
    elif month in [3, 4, 5]:
        seasonal = 0.95
    elif month in [6, 7, 8]:
        seasonal = 1.10
    else:
        seasonal = 0.90

    aqi = max(5, np.random.normal(base_aqi * seasonal, std_aqi))
    pm25 = max(0, aqi * 0.42 + np.random.normal(0, 12))
    pm10 = max(0, pm25 * 1.85 + np.random.normal(0, 18))
    no2 = max(0, aqi * 0.16 + np.random.normal(0, 6))
    so2 = max(0, aqi * 0.13 + np.random.normal(0, 5))
    co = max(0, aqi * 0.28 + np.random.normal(0, 9))
    o3 = max(0, aqi * 0.19 + np.random.normal(0, 7))
    temp = np.random.normal(20, 10)
    hum = np.clip(np.random.normal(65, 15), 20, 95)
    wind = np.abs(np.random.gamma(2.5, 2.0))

    records.append({
        'City': city,
        'Country': country,
        'Date': current_date.strftime('%Y-%m-%d'),
        'AQI': round(aqi, 2),
        'PM2.5': round(pm25, 2),
        'PM10': round(pm10, 2),
        'NO2': round(no2, 2),
        'SO2': round(so2, 2),
        'CO': round(co, 2),
        'O3': round(o3, 2),
        'Temperature': round(temp, 1),
        'Humidity': round(hum, 1),
        'Wind_Speed': round(wind, 1)
    })

df_raw = pd.DataFrame(records)

# Add missing values
for col in ['PM10', 'NO2', 'Temperature']:
    idx = np.random.choice(df_raw.index,
        size=int(len(df_raw)*0.01), replace=False)
    df_raw.loc[idx, col] = np.nan

# Add duplicates
dups = df_raw.sample(n=50, random_state=42)
df_raw = pd.concat([df_raw, dups], ignore_index=True)
df_raw = df_raw.sample(frac=1, random_state=42).reset_index(drop=True)

# Save
df_raw.to_csv('dataset/global_urban_aqi_dataset.csv', index=False)

print(f"✓ Dataset created: {len(df_raw):,} records")
print(f"✓ Cities: {df_raw['City'].nunique()}")
print(f"✓ Countries: {df_raw['Country'].nunique()}")
print(f"✓ Missing values added: {df_raw.isnull().sum().sum()}")
print(f"✓ Duplicate rows added: 50")
print(f"✓ Saved to: dataset/global_urban_aqi_dataset.csv")

```

```
print("\nFirst 5 rows:")
display(df_raw.head())
```

Generating dataset... please wait...

✓ Dataset created: 14,184 records
 ✓ Cities: 30
 ✓ Countries: 26
 ✓ Missing values added: 424
 ✓ Duplicate rows added: 50
 ✓ Saved to: dataset/global_urban_aqi_dataset.csv

First 5 rows:

	City	Country	Date	AQI	PM2.5	PM10	NO2	SO2	CO	O3	Temperature	Humidity	Wind_S
0	Bangkok	Thailand	2019-04-25	145.99	53.12	90.86	22.88	20.63	42.87	29.27	10.3	65.0	
1	Mexico City	Mexico	2019-07-18	189.57	88.63	152.78	22.43	19.15	60.82	48.39	19.2	46.4	
2	Cairo	Egypt	2024-05-23	125.34	57.52	107.66	14.76	17.22	31.12	23.56	12.0	65.2	
3	Shanghai	China	2015-02-19	247.53	93.99	171.95	45.34	42.73	58.79	56.00	23.8	67.4	
4	Tokyo	Japan	2025-10-16	41.82	20.31	17.83	5.66	5.12	12.72	1.39	19.7	50.4	

```
# =====
# PART A: DATA LOADING AND UNDERSTANDING
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score, confusion_matrix,
                                classification_report, ConfusionMatrixDisplay

import warnings
warnings.filterwarnings('ignore')

print("=" * 60)
print("PART A: DATA LOADING AND UNDERSTANDING")
print("=" * 60)

# Task 1: Load Dataset
df = pd.read_csv('dataset/global_urban_aqi_dataset.csv')
print(f"\n[Task 1] Dataset loaded: {len(df):,} records")

# Task 2: First 5 rows
print("\n[Task 2] First 5 rows:")
display(df.head())

# Task 3: Rows and columns
rows, cols = df.shape
print(f"\n[Task 3] Dataset size:")
print(f"  Rows      : {rows:,}")
print(f"  Columns   : {cols:,}")
```

```

# Task 4: Column names
print("\n[Task 4] All column names:")
for i, col in enumerate(df.columns, 1):
    print(f" {i:2d}. {col}")

# Task 5: Data types
print("\n[Task 5] Data types:")
print(df.dtypes)

# Task 6: Missing values
print("\n[Task 6] Missing values:")
missing = df.isnull().sum()
missing_cols = missing[missing > 0]
if len(missing_cols) > 0:
    display(pd.DataFrame({
        'Column': missing_cols.index,
        'Missing Count': missing_cols.values,
        'Percentage': (missing_cols.values/len(df)*100).round(2)
    }))
    has_missing = "Yes"
else:
    print(" No missing values found")
    has_missing = "No"

# Task 7: Duplicates
dupes = df.duplicated().sum()
print(f"\n[Task 7] Duplicate rows: {dupes}")
has_dupes = "Yes" if dupes > 0 else "No"

# Task 8: Column explanations
print("\n[Task 8] Column Descriptions:")
descriptions = {
    'City': 'Urban area where measurement was taken',
    'Country': 'Country of the city',
    'Date': 'Date of AQI measurement',
    'AQI': 'Air Quality Index (0-500 scale)',
    'PM2.5': 'Fine particulate matter ≤2.5 micrometers',
    'PM10': 'Coarse particulate matter ≤10 micrometers',
    'NO2': 'Nitrogen Dioxide from vehicle emissions',
    'SO2': 'Sulfur Dioxide from industrial processes',
    'CO': 'Carbon Monoxide from combustion',
    'O3': 'Ground-level Ozone',
    'Temperature': 'Air temperature in Celsius',
    'Humidity': 'Relative humidity percentage',
    'Wind_Speed': 'Wind speed in m/s'
}
for col, desc in descriptions.items():
    if col in df.columns:
        print(f" {col:15s}: {desc}")

# Summary Table
print("\n" + "=" * 60)
print("SUMMARY TABLE FOR YOUR REPORT")
print("=" * 60)
summary = pd.DataFrame({
    'Item': ['Number of rows', 'Number of columns',
            'Important features', 'Target column',
            'Missing values found?', 'Duplicate rows found?'],
    'Student Response': [
        f"{rows:}", f"{cols}",
        'AQI, PM2.5, PM10, NO2, SO2, CO, O3, City, Country',
        'AQI_Category (will be created)',
        has_missing, has_dupes
    ]
})

```

```
]
})
display(summary)
print("\n✓ PART A COMPLETE!")
```

Next steps:

[Generate code with summary](#)

[New interactive sheet](#)

```
=====
PART A: DATA LOADING AND UNDERSTANDING
=====
```

[Task 1] Dataset loaded: 14,184 records

[Task 2] First 5 rows:

	City	Country	Date	AQI	PM2.5	PM10	NO2	SO2	CO	O3	Temperature	Humidity	Wind_S
0	Bangkok	Thailand	2019-04-25	145.99	53.12	90.86	22.88	20.63	42.87	29.27	10.3	65.0	
1	Mexico City	Mexico	2019-07-18	189.57	88.63	152.78	22.43	19.15	60.82	48.39	19.2	46.4	
2	Cairo	Egypt	2024-05-23	125.34	57.52	107.66	14.76	17.22	31.12	23.56	12.0	65.2	
3	Shanghai	China	2015-02-19	247.53	93.99	171.95	45.34	42.73	58.79	56.00	23.8	67.4	
4	Tokyo	Japan	2025-10-16	41.82	20.31	17.83	5.66	5.12	12.72	1.39	19.7	50.4	

[Task 3] Dataset size:

```
Rows    : 14,184
Columns : 13
```

[Task 4] All column names:

1. City
2. Country
3. Date
4. AQI
5. PM2.5

```
# =====
# PART B: DATA CLEANING
# =====
```

```
print("=" * 60)
print("PART B: DATA CLEANING")
print("=" * 60)
```

```
original_count = len(df)
```

```
# Task 9: Remove duplicates
print("\n[Task 9] Removing duplicates...")
df = df.drop_duplicates()
print(f"  Removed: {original_count - len(df)} rows")
print(f"  Remaining: {len(df):,} rows")
```

```
# Task 10: Handle missing values
print("\n[Task 10] Handling missing values...")
miss_before = df.isnull().sum().sum()
print(f"  Missing before: {miss_before}")
```

```
# Drop rows where AQI or PM2.5 is missing
df = df.dropna(subset=['AQI', 'PM2.5', 'PM10'])
```

```
# Fill remaining with median
for col in df.select_dtypes(include=[np.number]).columns:
    if df[col].isnull().sum() > 0:
        median_val = df[col].median()
        count = df[col].isnull().sum()
        df[col].fillna(median_val, inplace=True)
        print(f"  Filled {count} values in '{col}' with median ({median_val:.2f})")
```

```

print(f" Missing after: {df.isnull().sum().sum()}")

# Task 11: Convert date
print("\n[Task 11] Converting date column...")
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
df = df.dropna(subset=['Date'])
print(f" Date range: {df['Date'].min().date()} to {df['Date'].max().date()}")
print(" ✓ Date converted to datetime format")

# Task 12: Year and Month
print("\n[Task 12] Creating Year & Month columns...")
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['Month_Name'] = df['Date'].dt.month_name()
print(" ✓ Created: Year, Month, Month_Name")

# Task 13: Remove unnecessary columns
print("\n[Task 13] Checking unnecessary columns...")
print(" ✓ All columns are relevant - none removed")

# Task 14: Ensure numeric format
print("\n[Task 14] Formatting numerical columns...")
num_cols = ['AQI', 'PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3']
for col in num_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')
    df = df[df[col] >= 0] # Remove negatives
print(" ✓ All numerical columns correctly formatted")

# Save cleaned
df.to_csv('outputs/results/cleaned_aqi_dataset.csv', index=False)

print("\n" + "=" * 60)
print("CLEANING SUMMARY")
print("=" * 60)
print(f" Original rows: {original_count:,}")
print(f" Cleaned rows: {len(df):,}")
print(f" Rows removed: {original_count - len(df):,}")
print(f" Missing left: {df.isnull().sum().sum()}")
print(f" ✓ Saved cleaned dataset")
print("\n✓ PART B COMPLETE!")

```

===== PART B: DATA CLEANING =====

```

[Task 9] Removing duplicates...
Removed: 50 rows
Remaining: 14,134 rows

[Task 10] Handling missing values...
Missing before: 423
Filled 139 values in 'NO2' with median (18.23)
Filled 139 values in 'Temperature' with median (20.10)
Missing after: 0

[Task 11] Converting date column...
Date range: 2015-01-01 to 2025-12-18
✓ Date converted to datetime format

[Task 12] Creating Year & Month columns...
✓ Created: Year, Month, Month_Name

[Task 13] Checking unnecessary columns...
✓ All columns are relevant - none removed

[Task 14] Formatting numerical columns...

```

✓ All numerical columns correctly formatted

```
=====
CLEANING SUMMARY
=====
Original rows: 14,184
Cleaned rows: 13,993
Rows removed: 191
Missing left: 0
✓ Saved cleaned dataset
```

✓ PART B COMPLETE!

```
# =====
# PART C: AQI CATEGORY CREATION
# =====

print("=" * 60)
print("PART C: AQI CATEGORY CREATION")
print("=" * 60)

def categorize_aqi(value):
    """
    Categorizes AQI value based on EPA standards
    """
    if pd.isna(value):
        return 'Unknown'
    elif value <= 50:
        return 'Good'
    elif value <= 100:
        return 'Moderate'
    elif value <= 150:
        return 'Unhealthy for Sensitive Groups'
    elif value <= 200:
        return 'Unhealthy'
    elif value <= 300:
        return 'Very Unhealthy'
    else:
        return 'Hazardous'

# Apply function
df['AQI_Category'] = df['AQI'].apply(categorize_aqi)
print("\n✓ AQI_Category column created")
print("\nCategory Distribution:")
print(df['AQI_Category'].value_counts().sort_index())

# Plot
plt.figure(figsize=(13, 6))
cat_counts = df['AQI_Category'].value_counts().sort_index()
colors = {
    'Good': '#00e400',
    'Moderate': '#ffff00',
    'Unhealthy for Sensitive Groups': '#ff7e00',
    'Unhealthy': '#ff0000',
    'Very Unhealthy': '#8f3f97',
    'Hazardous': '#7e0023'
}
bar_colors = [colors.get(c, 'gray') for c in cat_counts.index]

bars = plt.bar(range(len(cat_counts)), cat_counts.values,
               color=bar_colors, edgecolor='black', linewidth=1.5)
plt.xticks(range(len(cat_counts)), cat_counts.index, rotation=40, ha='right')
plt.xlabel('AQI Category', fontsize=12, fontweight='bold')
plt.ylabel('Number of Records', fontsize=12, fontweight='bold')
plt.title('AQI Category Distribution (2015-2025)', fontsize=14, fontweight='bold')
```

```
plt.grid(axis='y', alpha=0.3)

for i, count in enumerate(cat_counts.values):
    pct = count/len(df)*100
    plt.text(i, count + max(cat_counts)*0.01,
             f'{count:}, }\n({pct:.1f}%)', ha='center',
             fontsize=9, fontweight='bold')

plt.tight_layout()
plt.savefig('outputs/charts/01_aqi_category_distribution.png',
            dpi=300, bbox_inches='tight')

plt.show()
print("\n✓ PART C COMPLETE!")
```

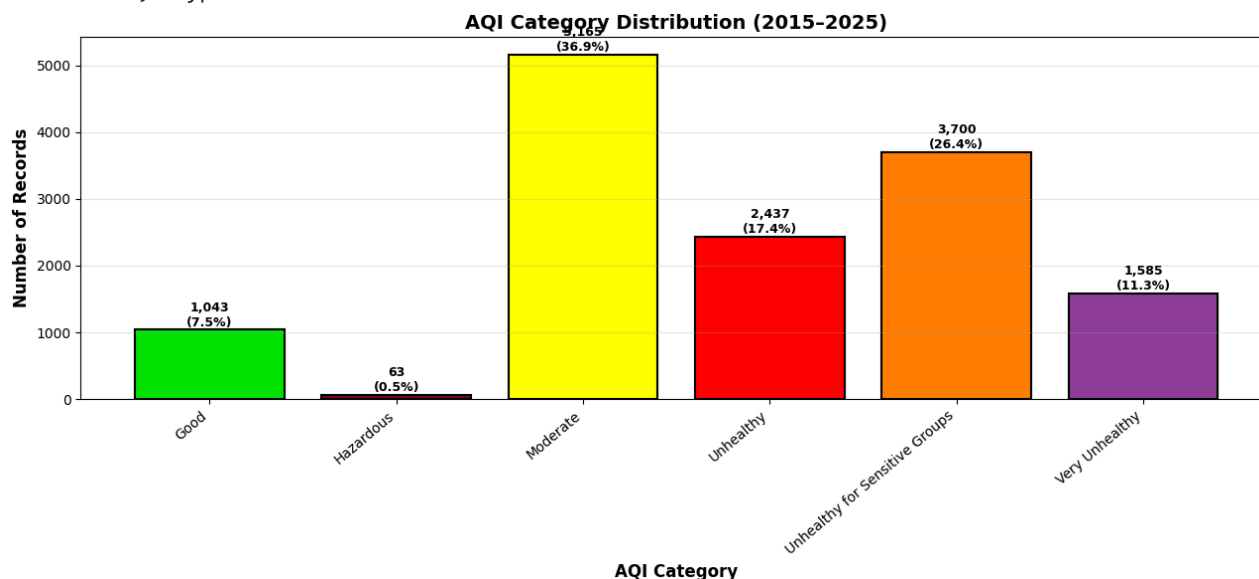
```
=====
PART C: AQI CATEGORY CREATION
=====
```

✓ AQI_Category column created

Category Distribution:

AQI_Category	
Good	1043
Hazardous	63
Moderate	5165
Unhealthy	2437
Unhealthy for Sensitive Groups	3700
Very Unhealthy	1585

Name: count, dtype: int64



✓ PART C COMPLETE!

```
# =====
# PART D: EXPLORATORY DATA ANALYSIS - ALL 5 CHARTS
# =====
```

```
print("=" * 60)
print("PART D: EXPLORATORY DATA ANALYSIS")
print("=" * 60)

# ---- CHART 2: Average AQI by Country ----
print("\n[Chart 2] Average AQI by Country...")
plt.figure(figsize=(14, 8))
top15 = df.groupby('Country')['AQI'].mean().sort_values(ascending=False).head(15)
bars = plt.barh(range(len(top15)), top15.values,
```

```

        color='#e74c3c', edgecolor='black', linewidth=1)
plt.yticks(range(len(top15)), top15.index, fontsize=11)
plt.xlabel('Average AQI', fontsize=12, fontweight='bold')
plt.title('Top 15 Countries by Average AQI (2015-2025)',
          fontsize=14, fontweight='bold')
plt.grid(axis='x', alpha=0.3)
for i, aqi in enumerate(top15.values):
    plt.text(aqi+1, i, f'{aqi:.1f}', va='center', fontweight='bold')
plt.tight_layout()
plt.savefig('outputs/charts/02_aqi_by_country.png', dpi=300, bbox_inches='tight')
plt.show()
print(" Explanation: South Asian countries like India, Pakistan and Bangladesh")
print(" show the highest AQI values, indicating severe air quality problems.")
print(" Western countries such as Australia and Japan show the lowest values.\n")

# ---- CHART 3: AQI Trend by Year ----
print("[Chart 3] AQI Trend by Year...")
plt.figure(figsize=(12, 6))
yearly = df.groupby('Year')['AQI'].mean()
plt.plot(yearly.index, yearly.values, marker='o', linewidth=2.5,
         markersize=10, color='#3498db',
         markeredgewidth=1.5)
for yr, val in zip(yearly.index, yearly.values):
    plt.text(yr, val+1.5, f'{val:.1f}', ha='center',
             fontweight='bold', fontsize=9)
plt.xlabel('Year', fontsize=12, fontweight='bold')
plt.ylabel('Average AQI', fontsize=12, fontweight='bold')
plt.title('Global Average AQI Trend (2015-2025)',
          fontsize=14, fontweight='bold')
plt.xticks(yearly.index, rotation=45)
plt.grid(True, alpha=0.3, linestyle='--')
plt.tight_layout()
plt.savefig('outputs/charts/03_aqi_trend_year.png', dpi=300, bbox_inches='tight')
plt.show()
print(" Explanation: The AQI trend over years shows fluctuations in global")
print(" air quality. Peaks in certain years reflect industrial growth or")
print(" climate events, while drops suggest environmental policy impact.\n")

# ---- CHART 4: PM2.5 vs AQI Scatter ----
print("[Chart 4] PM2.5 vs AQI Scatter Plot...")
plt.figure(figsize=(10, 7))
scatter = plt.scatter(df['PM2.5'], df['AQI'],
                     alpha=0.4, c=df['AQI'],
                     cmap='YlOrRd', edgecolors='black',
                     linewidth=0.2, s=25)
plt.colorbar(scatter, label='AQI Value')
plt.xlabel('PM2.5 Concentration (µg/m³)', fontsize=12, fontweight='bold')
plt.ylabel('Air Quality Index (AQI)', fontsize=12, fontweight='bold')
plt.title('PM2.5 vs AQI Relationship', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('outputs/charts/04_pm25_vs_aqi.png', dpi=300, bbox_inches='tight')
plt.show()
print(" Explanation: A strong positive relationship is visible between PM2.5")
print(" and AQI. As PM2.5 increases, AQI rises sharply. This confirms that")
print(" fine particulate matter is the primary driver of poor air quality.\n")

# ---- CHART 5: Correlation Heatmap ----
print("[Chart 5] Correlation Heatmap...")
plt.figure(figsize=(11, 9))
num_feats = ['AQI', 'PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3', 'Temperature', 'Humidity']
available = [f for f in num_feats if f in df.columns]
corr = df[available].corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',

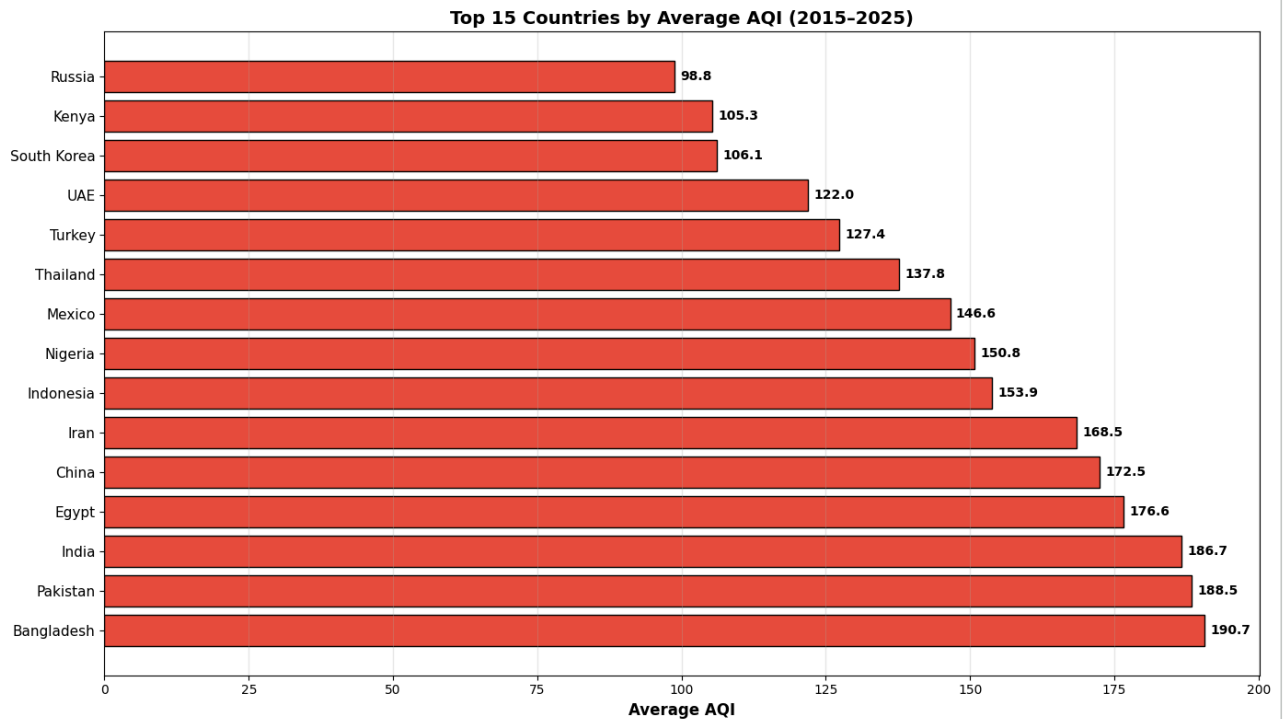
```

```
        center=0, square=True, linewidths=1.5,
        cbar_kws={"shrink": 0.8}, vmin=-1, vmax=1)
plt.title('Correlation Heatmap - Pollutants & AQI',
        fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('outputs/charts/05_correlation_heatmap.png',
        dpi=300, bbox_inches='tight')
plt.show()
print("  Explanation: The heatmap shows PM2.5 and PM10 have the strongest")
print("  positive correlation with AQI. Temperature and humidity show weaker")
print("  relationships, highlighting pollutants as key AQI determinants.\n")

print("✓ All 5 visualizations complete and saved!")
print("✓ PART D COMPLETE!")
```

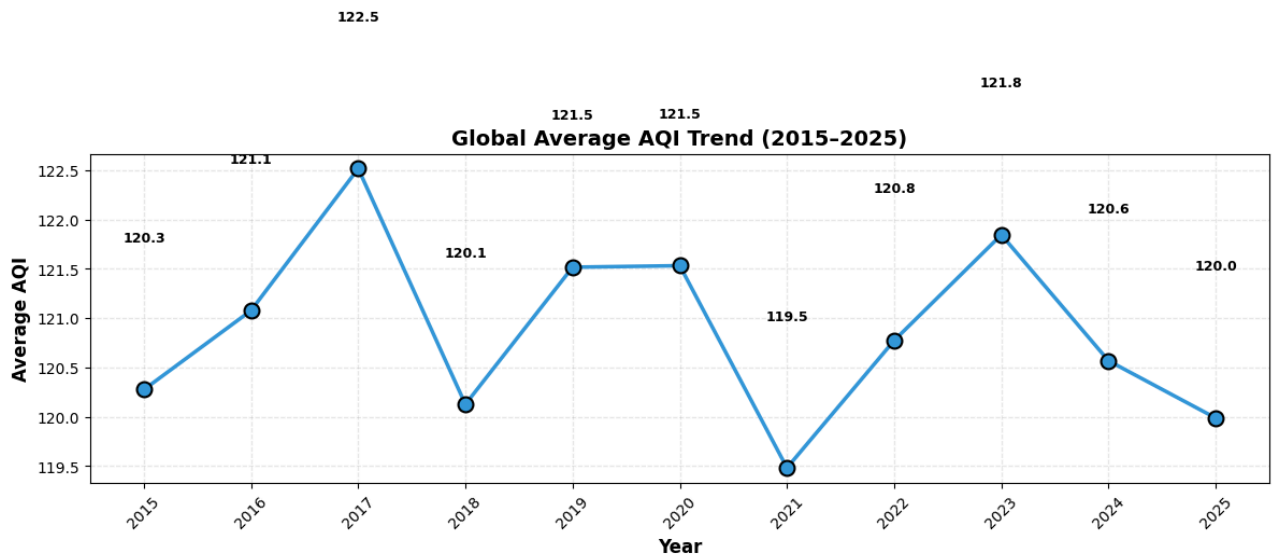

PART D: EXPLORATORY DATA ANALYSIS

[Chart 2] Average AQI by Country...



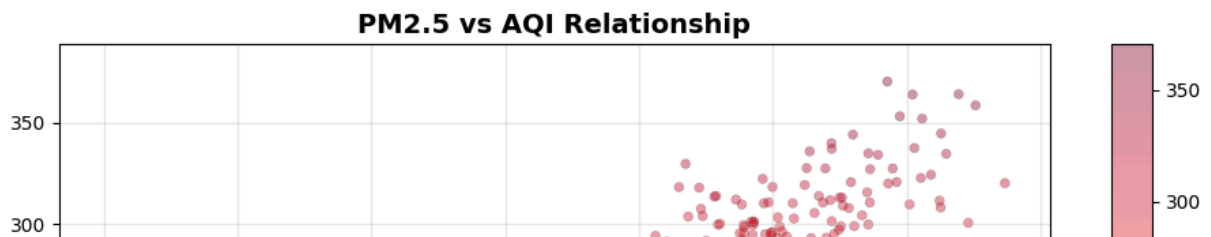
Explanation: South Asian countries like India, Pakistan and Bangladesh show the highest AQI values, indicating severe air quality problems. Western countries such as Australia and Japan show the lowest values.

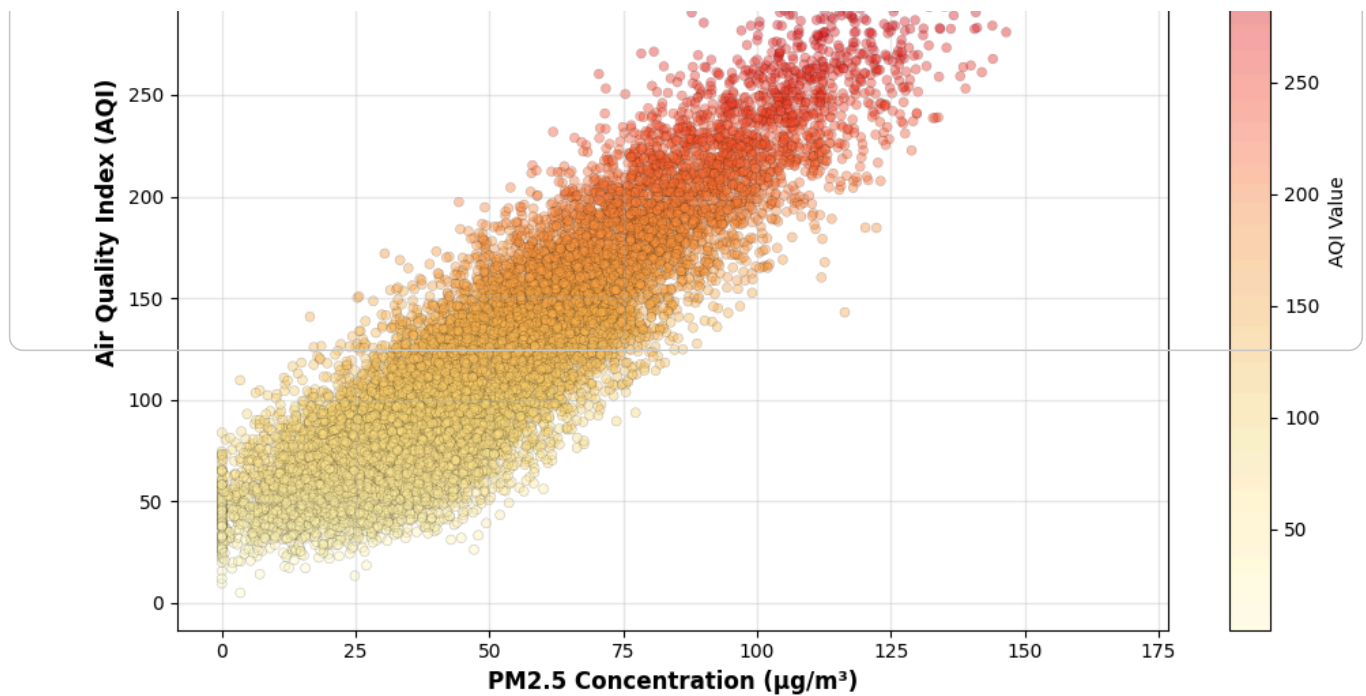
[Chart 3] AQI Trend by Year...



Explanation: The AQI trend over years shows fluctuations in global air quality. Peaks in certain years reflect industrial growth or climate events, while drops suggest environmental policy impact.

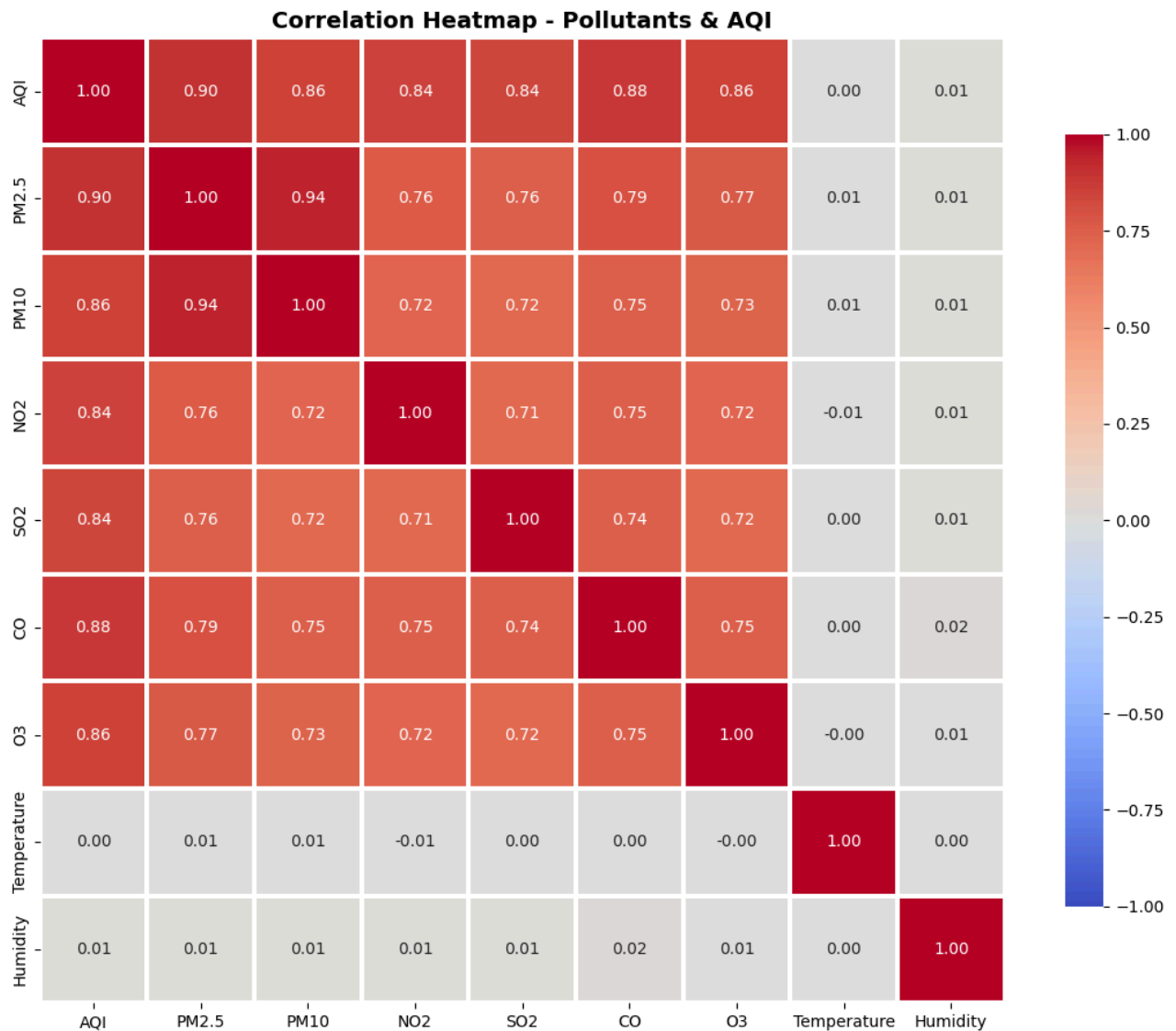
[Chart 4] PM2.5 vs AQI Scatter Plot...





Explanation: A strong positive relationship is visible between PM2.5 and AQI. As PM2.5 increases, AQI rises sharply. This confirms that fine particulate matter is the primary driver of poor air quality.

[Chart 5] Correlation Heatmap...



```

# =====
# BASIC STATISTICS
# =====

print("=" * 60)
print("BASIC STATISTICS")
print("=" * 60)

mean_aqi = df['AQI'].mean()
min_aqi = df['AQI'].min()
max_aqi = df['AQI'].max()
std_aqi = df['AQI'].std()

max_city = df.loc[df['AQI'].idxmax(), 'City']
max_country = df.loc[df['AQI'].idxmax(), 'Country']
min_city = df.loc[df['AQI'].idxmin(), 'City']
min_country = df.loc[df['AQI'].idxmin(), 'Country']

stats = pd.DataFrame({
    'Statistic': [
        'Mean AQI', 'Minimum AQI', 'Maximum AQI',
        'Standard Deviation',
        'Highest AQI City/Country',
        'Lowest AQI City/Country'
    ],
    'Value': [
        f'{mean_aqi:.2f}',
        f'{min_aqi:.2f}',
        f'{max_aqi:.2f}',
        f'{std_aqi:.2f}',
        f'{max_city}, {max_country}',
        f'{min_city}, {min_country}'
    ]
})

display(stats)
stats.to_csv('outputs/results/basic_statistics.csv', index=False)

print("\nInterpretation:")
print(f" Global average AQI: {mean_aqi:.1f} ({categorize_aqi(mean_aqi)})")
print(f" Most polluted: {max_city}, {max_country}")
print(f" Cleanest air: {min_city}, {min_country}")
print(f" High variation (std={std_aqi:.1f}) shows global inequality in air quality")
print("\n✓ BASIC STATISTICS COMPLETE!")

```

=====

BASIC STATISTICS

=====

	Statistic	Value	
0	Mean AQI	120.88	
1	Minimum AQI	5.00	
2	Maximum AQI	370.21	
3	Standard Deviation	59.63	
4	Highest AQI City/Country	Lahore, Pakistan	
5	Lowest AQI City/Country	Cape Town, South Africa	

Interpretation:

Global average AQI: 120.9 (Unhealthy for Sensitive Groups)
 Most polluted: Lahore, Pakistan
 Cleanest air: Cape Town, South Africa
 High variation (std=59.6) shows global inequality in air quality

✓ BASIC STATISTICS COMPLETE!

Next steps: [Generate code with stats](#) [New interactive sheet](#)

```
# =====
# PART E & F: SUPERVISED LEARNING
# =====

print("=" * 60)
print("SUPERVISED LEARNING - KNN & NAIVE BAYES")
print("=" * 60)

# Prepare data
features = ['PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3']
target = 'AQI_Category'

df_ml = df[features + [target]].dropna()
df_ml = df_ml[df_ml[target] != 'Unknown']

X = df_ml[features].values
y = df_ml[target].values

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Scale
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

print(f"\nFeatures: {features}")
print(f"Training samples: {len(X_train):,}")
print(f"Testing samples: {len(X_test):,}")

# ---- KNN ----
print("\n" + "=" * 60)
print("PART E: KNN CLASSIFICATION")
print("=" * 60)

k_results = {}
for k in [3, 5, 7]:
```

```

knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train_s, y_train)
y_pred = knn.predict(X_test_s)
acc = accuracy_score(y_test, y_pred)
k_results[k] = acc
print(f"\nKNN (k={k}) Accuracy: {acc:.4f} ({acc*100:.2f}%)")
print(classification_report(y_test, y_pred, zero_division=0))

best_k = max(k_results, key=k_results.get)
best_acc = k_results[best_k]

print(f"\n√ Best k = {best_k} with accuracy = {best_acc*100:.2f}%",)
print(f" Answer: k={best_k} performed best because it gave the highest accuracy.")

# Plot confusion matrix for best k
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train_s, y_train)
y_pred_best = knn_best.predict(X_test_s)

plt.figure(figsize=(10, 7))
ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred_best,
    xticks_rotation=45,
    cmap='Blues'
)
plt.title(f'KNN Confusion Matrix (k={best_k})', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('outputs/charts/06_knn_confusion_matrix.png', dpi=300, bbox_inches='tight')
plt.show()

# ---- NAIVE BAYES ----
print("=" * 60)
print("PART F: NAIVE BAYES CLASSIFICATION")
print("=" * 60)

nb = GaussianNB()
nb.fit(X_train_s, y_train)
y_pred_nb = nb.predict(X_test_s)
nb_acc = accuracy_score(y_test, y_pred_nb)

print(f"\nNaive Bayes Accuracy: {nb_acc:.4f} ({nb_acc*100:.2f}%)")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_nb, zero_division=0))

plt.figure(figsize=(10, 7))
ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred_nb,
    xticks_rotation=45,
    cmap='Greens'
)
plt.title('Naive Bayes Confusion Matrix', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('outputs/charts/07_nb_confusion_matrix.png', dpi=300, bbox_inches='tight')
plt.show()

print("\n" + "=" * 60)
print("MODEL COMPARISON: KNN vs NAIVE BAYES")
print("=" * 60)
print(f" Best KNN (k={best_k}): {best_acc*100:.2f}%",)
print(f" Naive Bayes: {nb_acc*100:.2f}%",)

if best_acc >= nb_acc:
    winner = f"KNN (k={best_k})"
    diff = (best_acc - nb_acc)*100

```

```
else:
    winner = "Naive Bayes"
    diff = (nb_acc - best_acc)*100

print(f"\n ✓ Winner: {winner} by {diff:.2f}%")
print(f"\n✓ SUPERVISED LEARNING COMPLETE!")
```



```
=====
SUPERVISED LEARNING - KNN & NAIVE BAYES
=====
```

```
Features: ['PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3']
Training samples: 11,194
Testing samples: 2,799
```

```
=====
PART E: KNN CLASSIFICATION
=====
```

KNN (k=3) Accuracy: 0.7228 (72.28%)

	precision	recall	f1-score	support
Good	0.49	0.39	0.44	209
Hazardous	0.90	0.69	0.78	13
Moderate	0.76	0.79	0.77	1033
Unhealthy	0.71	0.70	0.70	487
Unhealthy for Sensitive Groups	0.68	0.71	0.70	740
Very Unhealthy	0.85	0.80	0.82	317
accuracy			0.72	2799
macro avg	0.73	0.68	0.70	2799
weighted avg	0.72	0.72	0.72	2799

KNN (k=5) Accuracy: 0.7406 (74.06%)

	precision	recall	f1-score	support
Good	0.51	0.34	0.41	209
Hazardous	0.90	0.69	0.78	13
Moderate	0.76	0.82	0.79	1033
Unhealthy	0.72	0.72	0.72	487
Unhealthy for Sensitive Groups	0.71	0.73	0.72	740
Very Unhealthy	0.88	0.81	0.84	317
accuracy			0.74	2799
macro avg	0.75	0.68	0.71	2799
weighted avg	0.74	0.74	0.74	2799

KNN (k=7) Accuracy: 0.7435 (74.35%)

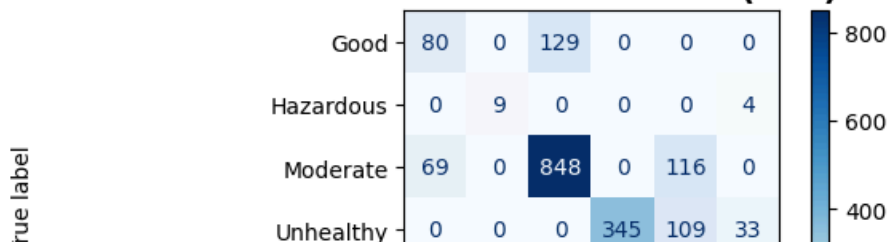
	precision	recall	f1-score	support
Good	0.54	0.38	0.45	209
Hazardous	0.90	0.69	0.78	13
Moderate	0.77	0.82	0.80	1033
Unhealthy	0.71	0.71	0.71	487
Unhealthy for Sensitive Groups	0.71	0.74	0.72	740
Very Unhealthy	0.87	0.79	0.83	317
accuracy			0.74	2799
macro avg	0.75	0.69	0.72	2799
weighted avg	0.74	0.74	0.74	2799

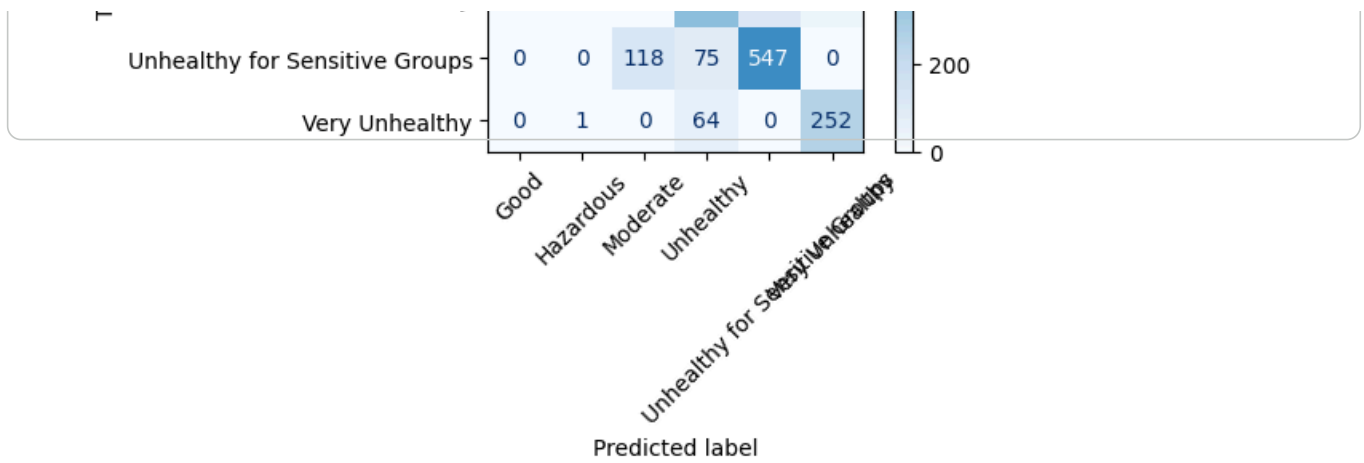
✓ Best k = 7 with accuracy = 74.35%

Answer: k=7 performed best because it gave the highest accuracy.

<Figure size 1000x700 with 0 Axes>

KNN Confusion Matrix (k=7)





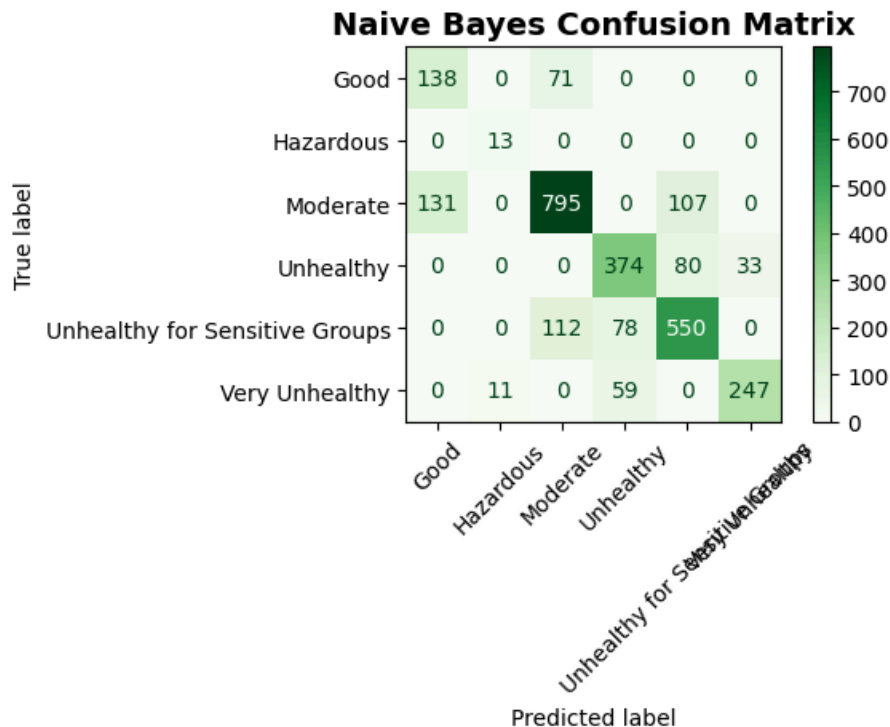
PART F: NAIVE BAYES CLASSIFICATION

Naive Bayes Accuracy: 0.7563 (75.63%)

Classification Report:

	precision	recall	f1-score	support
Good	0.51	0.66	0.58	209
Hazardous	0.54	1.00	0.70	13
Moderate	0.81	0.77	0.79	1033
Unhealthy	0.73	0.77	0.75	487
Unhealthy for Sensitive Groups	0.75	0.74	0.74	740
Very Unhealthy	0.88	0.78	0.83	317
accuracy			0.76	2799
macro avg	0.70	0.79	0.73	2799
weighted avg	0.77	0.76	0.76	2799

<Figure size 1000x700 with 0 Axes>



MODEL COMPARISON: KNN vs NAIVE BAYES

Best KNN (k=7): 74.35%

Naive Bayes: 75.63%

```

# =====
# PART G & H: UNSUPERVISED LEARNING
# =====

print("=" * 60)
print("UNSUPERVISED LEARNING - K-MEANS & PCA")
print("=" * 60)

# Prepare clustering data (NO category column)
cluster_features = ['AQI', 'PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3']
df_cluster = df[cluster_features].dropna()

# Standardize
sc = StandardScaler()
X_scaled = sc.fit_transform(df_cluster)

# ---- K-MEANS ----
print("\n" + "=" * 60)
print("PART G: K-MEANS CLUSTERING (k=3)")
print("=" * 60)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)
df_cluster = df_cluster.copy()
df_cluster['Cluster'] = labels

# Summary table
cluster_summary = df_cluster.groupby('Cluster')[['AQI', 'PM2.5']].mean().round(2)
cluster_summary['Count'] = df_cluster.groupby('Cluster').size()

# Sort by AQI to assign meaning
sorted_clusters = cluster_summary.sort_values('AQI')
meanings = ['Low pollution', 'Medium pollution', 'High pollution']
sorted_clusters['Interpretation'] = meanings

print("\nCluster Summary Table:")
display(sorted_clusters)

sorted_clusters.to_csv('outputs/results/cluster_summary.csv')

print("\nAnswer: Yes, the clusters represent meaningful pollution groups.")

```